

BASH SCRIPTING FUNDAMENTALS

Functions

Reusable building blocks and clean script structure

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: Function Foundations — 4 slides**
- 2 Section 2: Function Parameters — 4 slides**
- 3 Section 3: Scope And Local State — 4 slides**
- 4 Section 4: Return Values And Output — 4 slides**
- 5 Section 5: Main Pattern And Libraries — 4 slides**
- 6 Section 6: Error Logging And Assertions — 4 slides**
- 7 Section 7: Advanced Function Mechanics — 4 slides**
- 8 Section 8: Function Introspection — 4 slides**

9 Section 9: Manual Testing And Documentation — 4 slides

1 Section 10: Refactoring Into Functions — 4 slides

0

1 Section 11: Data Flow Techniques — 4 slides

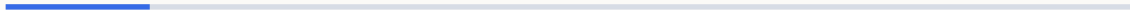
1

1 Section 12: Checkpoint Labs — 4 slides

2

1 Section 1

Function Foundations



1.1 Why functions · Name parentheses braces form

- Functions **name** a reusable piece of shell logic.
- They reduce repeated code in longer scripts.
- They create small units that are easier to test.
- The **name**() { } form is common Bash function syntax.
- A **space** is required before the opening brace body.
- Commands inside the braces run when the function is called.

Examples

```
# Repeated action
say_hi() { echo "hi $1"; }
say_hi Ada
# Named step
build() { make all; }
build
# Define
hello() { echo hello; }
hello() { echo hello; }
hello
```

1.2 Function keyword form · Calling functions

- Bash also accepts the **function** keyword.
- **It** is less portable but common in Bash-only scripts.
- Pick one style and use it consistently.
- Call a **function** by writing its name like a command.
- Arguments follow the name separated by spaces.
- The **function** must be defined before the call runs.

Examples

```
# Keyword
function hello { echo hello; }
# Keyword call
function build { echo build; }
build
# Simple call
greet() { echo hi; }
greet
greet() { echo "hi $1"; }
greet Sam
```


1.3 Declare before use · Function call status

- Bash executes scripts from top to bottom.
- A **function** definition must be seen before it is called.
- Place helper definitions near the top of scripts.
- A **function** returns the status of its last command by default.
- That status can drive if, &&, and ||.
- Use return when the last command is not the intended status.

Examples

```
# Before main
helper() { echo help; }
helper
# Main last
main() { helper; }
helper() { echo h; }
has_root() { grep -q root /etc/passwd; }
has_root && echo yes
exists() { [[ -e $1 ]]; }
exists file || echo missing
```

1.4 Compose small functions

- Small functions are easier to name and reuse.
- One function should do one clear job.
- **Compose** larger workflows from smaller helpers.

Examples

```
# Two helpers
clean() { rm -rf out; }
build() { mkdir out; }
# Workflow
run_all() { clean && build; }
run_all
```

2 Section 2

Function Parameters

2.1 First argument inside a function · All arguments with dollar at

- \$1 inside a function is the function **first** argument.
- **It** is separate from the script first argument.
- Quote it when using it as data.
- "\$@" expands to **all** function arguments safely.
- Each original argument stays a separate word.
- Use it when forwarding arguments to another command.

Examples

```
# Print first
show() { echo "$1"; }
show alpha
# Use path
size() { wc -c < "$1"; }
size file.txt
run() { command "$@"; }
run echo hi
show() { printf '%s\n' "$@"; }
show a "b c"
```

2.2 Argument count · Default arguments

- `$#` is the number of arguments passed to the function.
- Use it for arity checks.
- Check counts before reading required parameters.
- Parameter expansion can provide defaults.
- Defaults make optional parameters explicit.
- Use local variables to name defaulted values.

Examples

```
# Require one
need_one() { [[ $# -eq 1 ]] || return 2; }
# Print count
count_args() { echo "$#"; }
count_args a b
# Default env
deploy() { local env=${1:-dev}; echo "$env"; }
read_cfg() { local cfg=${1:-config.env}; echo "$cfg"; }
```

2.3 Shift inside functions · Validate function arguments

- **shift** removes the first function argument.
- **It** is useful after parsing an option or command.
- It does not **shift** the script arguments outside the function.
- Functions should check the inputs they require.
- Return a nonzero status when validation fails.
- Keep validation close to the top of the function.

Examples

```
# Drop command
run_cmd() { local cmd=$1; shift; "$cmd" "$@"; }
# Parse first
show_rest() { local first=$1; shift; printf '%s\n' "$@"; }
# Require file
read_file() { [[ -r $1 ]] || return 2; wc -l < "$1"; }
# Require int
twice() { [[ $1 =~ ^[0-9]+$ ]] || return 2; echo $(( $1*2 )); }
```


2.4 Usage function

- A **usage function** prints the valid command form.
- Keep it short enough to read after an error.
- Call it from argument validation branches.

Examples

```
# Usage text
usage() { echo "usage: app FILE"; }
# Use usage
[[ $# -eq 1 ]] || { usage; exit 2; }
```

3 Section 3

Scope And Local State

3.1 Local variables · Missing local pitfall

- **local** creates a variable scoped to the current function.
- It prevents accidental changes to global variables.
- Use **local** for temporary function state.
- Assignments without **local** can change global variables.
- This can create surprising behavior between functions.
- Declare scratch variables **local** by default.

Examples

```
# Local name
greet() { local name=$1; echo "hi $name"; }
# Local count
count() { local n=0; ((n++)); echo $n; }
# Global leak
tmp=old
set_tmp() { tmp=new; }
set_tmp() { local tmp=new; echo "$tmp"; }
```

3.2 Readonly function variables · Global variables are shared state

- **readonly** prevents reassignment after a value is set.
- Inside functions, combine local and **readonly** for constants.
- Use it for values that must not change within a helper.
- **Global** variables are visible inside functions.
- They couple helpers to outer script state.
- Prefer parameters and outputs for most data flow.

Examples

```
# Local readonly
show() { local -r path=$1; echo "$path"; }
# Readonly global
readonly APP_NAME=demo
# Read global
prefix=app
name() { echo "$prefix-$1"; }
set_env() { local env=$1; echo "$env"; }
```

3.3 Local arrays · Nameref intro

- Arrays can be **local** to a function.
- Use **local** -a for indexed arrays.
- Quote array expansions when passing values along.
- local -n creates a **nameref** to another variable.
- It lets a function update a caller variable by name.
- Use it carefully because it increases coupling.

Examples

```
# Local array
list() { local -a xs=(a b); printf '%s\n' "${xs[@]}"; }
# Build array
make_args() { local -a args=(--name "$1"); printf '%s\n' "${args[@]}"; }
# Set by name
set_value() { local -n ref=$1; ref=$2; }
set_value result ok
first_item() { local -n arr=$1; echo "${arr[0]}"; }
```

3.4 Unset temporary names

- **unset** removes a variable name from the current scope.
- Local variables disappear automatically when a function returns.
- Use **unset** when a long function must clear sensitive values early.

Examples

```
# Unset local
demo() { local token=$1; unset token; }
# Unset global
cache=value
unset cache
```

4 Section 4

Return Values And Output

4.1 Return status · Return status range

- **return** sets the function exit status.
- A status of 0 means success.
- Use nonzero statuses for validation or command failures.
- Shell return statuses are small integers.
- Do not use **return** for strings or structured data.
- Use stdout for data a caller should capture.

Examples

```
# Return success
ok() { return 0; }
ok && echo yes
# Return failure
fail() { return 1; }
fail || echo no
# Bad data return
get_name() { echo Ada; }
is_file() { [[ -f $1 ]]; }
```

4.2 Return versus echo · Capture command substitution

- **return** communicates success or failure.
- `echo` or `printf` communicates textual output.
- Keep status and data roles separate.
- Command substitution captures stdout from a function.
- Trailing newlines are removed by the substitution.
- Quote the assignment result when using it later.

Examples

```
# Predicate
is_prod() { [[ $1 == prod ]]; }
# Producer
make_name() { printf 'app-%s\n' "$1"; }
# Capture
now() { date +%s; }
ts=$(now)
slug() { echo "app-$1"; }
name=$(slug api)
```

4.3 Stdout versus stderr · Quiet predicate functions

- **Stdout** is for data a caller may capture.
- **Stderr** is for diagnostics and errors.
- Send messages to stderr when **stdout** must stay parseable.
- Predicate functions often print nothing.
- Their status is the answer.
- This makes them easy to use in if statements.

Examples

```
# Error output
warn() { echo "warn: $" ">&2; }
# Data output
value() { printf '%s\n' "$1"; }
# Has command
has_cmd() { command -v "$1" >/dev/null 2>&1; }
# Readable
can_read() { [[ -r $1 ]]; }
```

4.4 Multiple values with echo

- A function can print **multiple** lines as output.
- The caller must parse that output deliberately.
- For complex data, prefer clear formats or separate helpers.

Examples

```
# Two lines
pair() { printf '%s\n' name Ada; }
# Parse simple
vals=$(pair)
printf '%s\n' "$vals"
```

5 Section 5

Main Pattern And Libraries

5.1 Main function pattern · Pass arguments to main

- A **main** function names the script entry point.
- It keeps top-level code short.
- It makes script flow easier to scan.
- Use "\$@" when calling **main** from top level.
- This preserves the original script arguments.
- **Main** can then validate and dispatch them.

Examples

```
# Main
main() { echo run; }
main "$@"

# Main status
main() { return 0; }
main "$@"

main() { printf '%s\n' "$@"; }
main "$@"

main() { echo "cmd=$1"; }
main "$@"
```

5.2 Sourcing a library · Guard against re source

- source reads another shell file into the current shell.
- Functions defined there become available to the script.
- Use relative paths carefully when **sourcing** libraries.
- A **guard** variable can prevent loading a library twice.
- Set the **guard** after the first successful load.
- This protects repeated readonly declarations.

Examples

```
# Source lib
source ./lib.sh
helper
# Dot form
. ./lib.sh
helper
[[ ${LIB_LOADED:-} ]] && return 0
LIB_LOADED=1
[[ ${UTILS_LOADED:-} ]] && return 0
readonly UTILS_LOADED=1
```

5.3 Function library layout · Helpers before main

- Group related helper functions in a small library file.
- Keep script entry logic out of reusable libraries.
- Use clear prefixes when many libraries are sourced.
- Place helper definitions before main calls them.
- This matches Bash top to bottom execution.
- The main call should usually be near the end.

Examples

```
# Utils file
log_info() { echo "info: $*"; }
# Script use
source ./utils.sh
log_info start
# Order
helper() { echo h; }
main() { helper; }
main() { echo run; }
main "$@"
```

5.4 Run only when executed

- BASH_SOURCE can distinguish source from execute.
- This allows a file to be both script and library.
- Call main only when the file is executed directly.

Examples

```
# Entry guard
if [[ ${BASH_SOURCE[0]} == $0 ]]; then main "$@"; fi
# Source safe
main() { echo run; }
[[ ${BASH_SOURCE[0]} == $0 ]] && main "$@"
```

6 Section 6

Error Logging And Assertions

6.1 Die helper · Error helper

- A **die helper** prints an error and exits.
- It centralizes fatal **error** formatting.
- Use it only where exiting the script is intended.
- An **error helper** prints to stderr without exiting.
- It lets callers decide whether to return or continue.
- **This** is useful inside reusable functions.

Examples

```
# Die
die() { echo "error: $" >&2; exit 1; }
# Use die
[[ -r $cfg ]] || die "missing config"
# Error
error() { echo "error: $" >&2; }
# Return after error
need_file() { [[ -f $1 ]] || { error missing; return 2; }; }
```


6.2 Info logging helper · Warn logging helper

- An **info** helper marks normal progress messages.
- Send logs to stderr when stdout is data.
- Keep the format consistent across scripts.
- A **warn** helper marks nonfatal problems.
- It should not exit on its own.
- Use it when the script can continue safely.

Examples

```
# Info
info() { echo "info: $" >&2; }
# Use info
info "starting build"
# Warn
warn() { echo "warn: $" >&2; }
# Use warn
[[ -f optional.env ]] || warn "no optional env"
```

6.3 Assert command helper · Assert file helper

- An **assert** helper checks an invariant during a script.
- Failed assertions should return or exit clearly.
- Use assertions for programmer expectations, not user input.
- A file assertion names a common required condition.
- It keeps call sites short and readable.
- Return nonzero rather than exiting inside reusable libraries.

Examples

```
# Assert
assert() { "$@" || die "assert failed: $*"; }
# Assert use
assert test -d src
# Require file
require_file() { [[ -f $1 ]] || return 2; }
# Use require
require_file config.env || die "need config"
```

6.4 Command wrapper

- A wrapper function adds policy around another **command**.
- It can add logging, defaults, or validation.
- Forward arguments with "\$@" to preserve words.

Examples

```
# Git wrapper
git_checked() { git "$@" || die "git failed"; }
# Curl wrapper
fetch() { curl -fsS "$@"; }
```

7 Section 7

Advanced Function Mechanics

7.1 Functions and exit · Set e inside functions

- `exit` inside a function exits the shell or script.
- `return` exits only the function.
- Library **functions** should usually `return` instead of `exit`.
- **set** -e can interact with function calls in surprising ways.
- Commands tested by `if` or `||` are treated specially.
- Use explicit status handling for important failures.

Examples

```
# Return
check() { [[ -e $1 ]] || return 2; }
# Exit fatal
fatal() { echo bad >&2; exit 1; }
# Explicit return
copy() { cp "$1" "$2" || return; }
# If tested
if risky_fn; then echo ok; fi
```

7.2 Err trap caution · Recursion caution

- **ERR** traps can run when commands fail under set -e.
- Functions make trap flow harder to reason about.
- Keep trap based error handling small and tested.
- Bash functions can call themselves recursively.
- Deep **recursion** is usually a poor fit for shell scripts.
- Prefer loops for repeated shell work.

Examples

```
# Err trap
trap 'echo failed >&2' ERR
set -E
# Function failure
work() { false; }
work
# Recursive shape
countdown() { (( $1 == 0 )) || countdown $(( $1 - 1 )); }
for (( n=3; n>=0; n-- )); do echo $n; done
```

7.3 Overriding commands · Builtin command bypass

- A function can have the same name as a command.
- This changes what unqualified calls run.
- **Use** command to bypass a function wrapper.
- **The** command builtin skips shell function lookup.
- **It** is useful inside wrappers that call the original command.
- It can also check external command availability.

Examples

```
# Wrapper
ls() { command ls --color=auto "$@"; }
# Bypass
command ls
# Call original
grep() { command grep --color=auto "$@"; }
# Find command
command -v bash >/dev/null
```


7.4 Unset function

- **unset** -f removes a function definition.
- **It** is useful in interactive shells and tests.
- Avoid removing shared helpers during normal script flow.

Examples

```
# Remove function
tmp_fn() { echo tmp; }
unset -f tmp_fn
# Replace wrapper
unset -f ls
```

8 Section 8

Function Introspection

8.1 Declare $f \cdot$ Type t function

- **declare** -f prints function definitions.
- **It** is useful for debugging sourced libraries.
- Use a function name to inspect one definition.
- **type** -t reports what kind of name Bash resolves.
- It prints function for function names.
- **It** is useful before overriding or dispatching names.

Examples

```
# Show function
hello() { echo hi; }
declare -f hello
# All functions
declare -f
# Check type
hello() { ;; }
type -t hello
type -t cd
```

8.2 Command V · Declare F

- **command** -V describes how a name will be interpreted.
- It can show functions, builtins, aliases, and files.
- Use it when debugging resolution problems.
- **declare** -F lists function names without bodies.
- **It** is shorter than declare -f for inventories.
- Use it to confirm a library loaded expected helpers.

Examples

```
# Describe
hello() { ;; }
command -V hello
# Describe ls
command -V ls
# List names
declare -F
declare -F helper >/dev/null && echo loaded
```

8.3 Exported functions caution · Readonly functions

- export -f can pass functions to child Bash processes.
- **It** is Bash specific and should be used sparingly.
- Prefer script files for reusable child process behavior.
- **readonly** -f prevents a function from being redefined or unset.
- It can protect critical helpers in controlled scripts.
- Use it sparingly because it makes tests harder.

Examples

```
# Export function
helper() { echo h; }
export -f helper
# Child bash
bash -c helper
# Protect
die() { exit 1; }
readonly -f die
readonly -f
```

8.4 Namespace prefixes

- Prefixes reduce function name collisions.
- **They** are helpful in sourced libraries.
- Use consistent names such as `log_info` or `path_join`.

Examples

```
# Prefixed log
log_info() { echo "info: $*"; }
# Prefixed path
path_join() { echo "$1/$2"; }
```


9 Section 9

Manual Testing And Documentation

9.1 Manual function calls · Interactive source testing

- Source a file and call functions directly while developing.
- This shortens the edit and test loop.
- Use simple arguments that cover success and failure.
- An **interactive** shell can load your helpers with source.
- declare -f confirms the definitions are present.
- unset -f can clean up test definitions.

Examples

```
# Source then call
source ./script.sh
my_fn test
# Call failure
my_fn missing || echo failed
# Load
source ./lib.sh
declare -F
unset -f helper
```

9.2 Temporary argument tests · Capture output tests

- Small test calls can exercise argument handling.
- Include spaces in sample arguments.
- Verify both output and status.
- Command substitution can check function output.
- Compare captured output to the expected value.
- Quote variables in test comparisons.

Examples

```
# Space arg
show() { printf '<%s>\n' "$@"; }
show "a b"

# Status arg
need_one() { [[ $# -eq 1 ]]; }
need_one a b || echo bad
got=$(slug api)
[[ $got == app-api ]]
[[ $got == ok ]] || echo "got $got"
```

9.3 Compare status tests · ShellCheck SC2155 note

- Status tests check whether predicates succeed or fail.
- Use if or && and || for manual checks.
- Avoid reading \$? when direct conditionals are clearer.
- SC2155 warns about declaring and assigning in one command.
- Separate declaration from assignment when status matters.
- **This** is especially useful with command substitution.

Examples

```
# Predicate pass
is_num() { [[ $1 =~ ^[0-9]+$ ]]; }
is_num 42 && echo pass
# Predicate fail
is_num x || echo fail
# Separated local
local value
value=$(cmd)
local name=$1
```

9.4 Documentation comments

- A short comment can describe a function contract.
- Mention inputs, output, and status when helpful.
- Keep comments close to the function definition.

Examples

```
# Contract
# Prints a slug for one name.
slug() { echo "app-$1"; }
# Status comment
# Returns 0 when the path is readable.
can_read() { [[ -r $1 ]]; }
```


10

Section 10

Refactoring Into Functions

10.1 Extract repeated code · Pure functions

- **Repeated** command sequences are good function candidates.
- Name the extracted behavior by its purpose.
- Keep the call site shorter after extraction.
- A **pure** style uses parameters and stdout without hidden state.
- **It** is easier to test and reuse.
- Shell functions are often not perfectly **pure**, but the idea helps.

Examples

```
# Extract
backup() { cp "$1" "$1.bak"; }
# Reuse
backup app.conf
backup db.conf
# Pure slug
slug() { printf '%s\n' "${1// /-}"; }
name=$(slug "api server")
```

10.2 Side effect functions · Limit global dependencies

- **Side effect functions** change files, directories, or process state.
- Name them with verbs so the change is obvious.
- Validate inputs before making changes.
- Functions that rely on globals are harder to reuse.
- Pass paths and options as arguments instead.
- Keep unavoidable globals readonly when possible.

Examples

```
# Make directory
ensure_dir() { [[ -d $1 ]] || mkdir -p "$1"; }
# Remove file
remove_file() { [[ -f $1 ]] && rm "$1"; }
# Pass path
load_cfg() { source "$1"; }
# Avoid hidden path
run_tool() { local bin=$1; shift; "$bin" "$@"; }
```

10.3 Small predicate helpers · Wrap pipelines

- Predicate helpers **wrap** common tests behind clear names.
- They should return status and usually print nothing.
- This makes if statements read like intent.
- A pipeline can be hidden behind a named function.
- The name documents what the pipeline produces.
- Be clear whether stdout is data or logs.

Examples

```
# Predicate
is_readable_file() { [[ -f $1 && -r $1 ]]; }
# Use predicate
if is_readable_file config; then echo ok; fi
# Count errors
count_errors() { grep -c ERROR "$1"; }
# Top names
top_names() { cut -d, -f1 "$1" | sort; }
```

10.4 Teardown helpers

- Cleanup code is a good candidate for a helper.
- A named cleanup function can be used with trap.
- Keep cleanup safe to call more than once.

Examples

```
# Cleanup
cleanup() { rm -rf "$tmpdir"; }
# Trap cleanup
trap cleanup EXIT
```

11

Section 11

Data Flow Techniques

11.1 Printf v assignment · Nameref output

- **printf** -v assigns formatted text to a variable by name.
- It avoids command substitution for simple formatting.
- Use it when building strings inside functions.
- A **nameref** can act like an output parameter.
- The caller passes the variable name to update.
- Document this clearly because it is less obvious than stdout.

Examples

```
# Assign formatted
printf -v name 'app-%s' "$1"
# Inside function
make_name() { printf -v result 'app-%s' "$1"; }
# Output variable
make_name() { local -n out=$1; out=app; }
make_name result
find_cfg() { local -n out=$1; out=config.env; }
```

11.2 Global output discouraged · Arrays passed by name

- Writing results into globals couples caller and function.
- It can be acceptable in tiny scripts but scales poorly.
- Prefer stdout or nameref outputs for reusable helpers.
- Bash **arrays** are often passed to functions by variable name.
- `local -n` lets the function read or update that array.
- Avoid using the same nameref name as the caller array.

Examples

```
# Global result
RESULT=
set_result() { RESULT=ok; }
# Stdout result
get_result() { echo ok; }
# Read array
first() { local -n a=$1; echo "${a[0]}"; }
add_item() { local -n a=$1; a+=("$2"); }
```


11.3 Read loop in function · Mapfile in function

- A function can own a while **read** loop.
- Pass the input path as an argument.
- Keep the loop output format stable.
- **mapfile** can load function input into a local array.
- Use -t to remove trailing newlines.
- **This** is useful when multiple passes over lines are needed.

Examples

```
# Print lines
print_file() { while IFS= read -r l; do echo "$l"; done < "$1"; }
# Count lines
count_file() { local n=0; while read -r _; do ((n++)); done < "$1"; echo $n; }
# Local mapfile
load_lines() { local -a lines; mapfile -t lines < "$1"; echo "${#lines[@]}"; }
# Print first
first_line() { local -a l; mapfile -t l < "$1"; echo "${l[0]}"; }
```

11.4 Stable output format

- Function output becomes an interface for callers.
- Changing columns or separators can break scripts.
- Use printf for predictable formatting.

Examples

```
# Key value
emit_pair() { printf '%s=%s\n' "$1" "$2"; }
# One per line
emit_list() { printf '%s\n' "$@"; }
```

1 2

Section 12

Checkpoint Labs

12.1 Build usage lab · Refactor script lab

- Write a usage function for one script command.
- Call it when arguments are missing or invalid.
- Return status 2 for usage errors.
- Find two repeated command groups in a script.
- Extract each group into a named function.
- Call the new functions from main.

Examples

```
# Usage
usage() { echo "usage: app build|test"; }
# Usage error
[[ $# -gt 0 ]] || { usage; exit 2; }
# Extract build
build_app() { make build; }
# Main uses
main() { build_app && test_app; }
```

12.2 Validator lab · Logging library lab

- Create predicate functions for common input checks.
- Keep predicate output quiet.
- Use the predicates in if statements.
- Create info, warn, and error helpers.
- Send all log messages to stderr.
- Use a consistent prefix for each severity.

Examples

```
# Integer predicate
is_int() { [[ $1 =~ ^-[0-9]+$ ]]; }
# Path predicate
is_dir() { [[ -d $1 ]]; }
# Info warn
info() { echo "info: $" >&2; }
warn() { echo "warn: $" >&2; }
error() { echo "error: $" >&2; }
```

12.3 Retry function lab · Dispatch function lab

- Wrap **retry** behavior in a function.
- Accept the command and its arguments with "\$@".
- Return failure after the final attempt.
- Use case inside main to choose a function.
- **Keep** command names separate from implementation details.
- Route unknown commands to usage.

Examples

```
# Retry shape
retry() { for i in 1 2 3; do "$@" && return 0; done; return 1; }
# Use retry
retry curl -fsS http://localhost
# Dispatch
case $1 in build) build ;; test) test_all ;; *) usage ;; esac
build() { echo build; }
test_all() { echo test; }
```

12.4 Function checklist

- Name the **function** by the job it performs.
- Declare local variables for temporary state.
- Separate status, logs, and data output deliberately.

Examples

```
# Checklist helper
has_file() { [[ -f $1 ]]; }
# Checklist output
make_slug() { printf '%s\n' "${1// /-}"; }
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory

04 · Functions

